Lesson 2 Teacher's Key

Unreal Engine (optional advanced track)

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

Requires Setup's full project configuration already done: Lumen off, default Light/Sky actors deleted, BP_Player created (parent Static Mesh Actor) with its Static Mesh set to Cube and M_Player_Orange (Unlit, a Texture Sample from the imported player_circle_orange.png wired to Emissive Color - the real sprite, not a flat color) applied, Mobility changed from Static to Movable on BP_Player's class default, Auto Receive Input set to Player 0 in BP_Player's Class Defaults, BP_NoPawnGameMode set as GameMode Override (Default Pawn Class = None), the Orthographic Camera placed and Auto-Activated for Player 0, and IA_MoveRight/Left/Up/Down + IMC_Default (all 4 keys mapped) built, with IMC_Default applied via Add Mapping Context in BP_Player's own BeginPlay. Watch for students who forget Auto Receive Input (the #1 miss now - a plain Actor Blueprint never receives keyboard input without it, and Add Actor World Offset's Target pin can be left unwired throughout, since inside BP_Player's own graph that correctly defaults to Self), who leave the Enhanced Input Action Event on its default Started pin instead of Triggered (moves once per press instead of gliding while held), who forget to right-click Delta Location and choose Split Struct Pin before trying to wire a value into just X or Y, or who forget BP_Player's Mobility (a Static component silently ignores every movement call). BP_Player's own Event Graph:

```
Enhanced Input Action Event (IA_MoveRight), Triggered -> Add Actor World
Offset(Target unwired = Self, DeltaLocation = (X = 300 * Get World Delta
Seconds, Y=0, Z=0))
```

Level 2 - Complete Worked Solution

BP_Player's own Event Graph, 4 Enhanced Input Action Event nodes (one per Input Action, all on their Triggered pin) each feeding an Add Actor World Offset node with Target left unwired (defaults to Self), Delta Location scaled by Get World Delta Seconds x Speed (300) on every direction so movement stays smooth and framerate-independent:

```
Enhanced Input Action Event (IA_MoveRight), Triggered -> Add Actor World
Offset(Self, DeltaLocation = (X = 300 * Get World Delta Seconds, Y=0,
Z=0))
Enhanced Input Action Event (IA_MoveLeft),  Triggered -> Add Actor World
Offset(Self, DeltaLocation = (X = -300 * Get World Delta Seconds, Y=0,
Z=0))
Enhanced Input Action Event (IA_MoveUp),    Triggered -> Add Actor World
Offset(Self, DeltaLocation = (X=0, Y = 300 * Get World Delta Seconds,
Z=0))
Enhanced Input Action Event (IA_MoveDown),  Triggered -> Add Actor World
Offset(Self, DeltaLocation = (X=0, Y = -300 * Get World Delta Seconds,
Z=0))
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking (or if a student blows past all of them, this is your own jumping-off point for exploring further right alongside them).

Animate: blink while moving, static when idle

Toggling visibility is the Blueprint equivalent of the counter-based blink used in the code-path engines - since BP_Player's mesh is a 3D Shape rather than a 2D sprite, Set Actor Hidden In Game is the closest match to SpriteRenderer.enabled or Godot's visible. This needs an IsMoving state that's true while any direction key is held, set from the Started/Completed pins of the 4 Enhanced Input Action Events - see the Structure idea below for why it's split this way:

```
Event Tick ->
  Branch (IsMoving)
    True:  BlinkTimer += DeltaSeconds
           Branch (BlinkTimer >= 0.1) -> True: Blink = NOT Blink;
BlinkTimer = 0
           Set Actor Hidden In Game(Self, Blink)
    False: Set Actor Hidden In Game(Self, false)
           BlinkTimer = 0
```

Sound: repeating beep-beep while held, silent when idle

Play Sound 2D is fire-and-forget - it spawns a sound with no handle to check whether it's still playing, unlike Unity's AudioSource.isPlaying or Godot's AudioStreamPlayer2D.playing. To get the same repeating "beep-beep while held" rhythm those engines use (rather than one blip per press), gate it behind a SoundCooldown float counted down by Get World Delta Seconds on the Triggered pin - the same cooldown-timer idiom used for fire-rate limiting elsewhere in this course:

```
Enhanced Input Action Event (IA_MoveRight), Triggered ->
  Add Actor World Offset(Self, DeltaLocation = (X = 300 * Get World Delta
Seconds, Y=0, Z=0))
  SoundCooldown -= Get World Delta Seconds
  Branch (SoundCooldown <= 0) -> True: Play Sound 2D(sfx_blip.wav); Set
SoundCooldown = 0.15
```

Recolor: different material per direction, reset when idle

Import player_circle_red.png, player_circle_blue.png, player_circle_green.png, and player_circle_yellow.png (this lesson's Assets folder) and build one Unlit Material per color the same way Setup built M_Player_Orange - a Texture Sample from that PNG wired straight into Emissive Color. Each color is its own separate, pre-made Material (the real supplied sprite, not a flat color standing in for it), swapped onto BP_Player with a Set Material node off the Started pin - one swap per press-down is all that's needed, no reason to re-swap every frame while held. If Set Material doesn't appear in the search, call Get Static Mesh Component first (Target left unwired, defaults to Self), or uncheck "Context Sensitive" in the search popup - a known Blueprint search quirk. Same Idle/Moving concept as the State Diagram from this lesson's bonus section:

```
Enhanced Input Action Event (IA_MoveRight), Started -> Set Material(Self,
Material Index 0, M_Player_Red)
Enhanced Input Action Event (IA_MoveLeft),  Started -> Set Material(Self,
Material Index 0, M_Player_Blue)
```

```
Enhanced Input Action Event (IA_MoveUp),    Started -> Set Material(Self,
Material Index 0, M_Player_Green)
Enhanced Input Action Event (IA_MoveDown),  Started -> Set Material(Self,
Material Index 0, M_Player_Yellow)
Event Tick, Branch(IsMoving) False -> Set Material(Self, Material Index 0,
M_Player_Orange)  // idle
```

Clamp movement to a play area

After each direction's Triggered-driven Add Actor World Offset, chain: Get Actor Location -> Break Vector -> Clamp (Float) on X and Y against your chosen min/max (half the Camera's Ortho Width from Setup is a reasonable starting bound) -> Make Vector -> Set Actor Location. Clamping after the fact is simpler to explain than pre-checking bounds, and is a common real pattern in Blueprint movement code. Because movement now runs every frame while held, this check also re-runs every frame while held, keeping BP_Player inside a defined rectangle no matter how long a key stays down.

Structure: group everything triggered by the same action in one place

Unreal Blueprint genuinely has three different execution contexts here, and that's correct, not a mistake to avoid: each direction's movement, Clamp check, and cooldown-gated sound all chain off that direction's own Enhanced Input Action Event's Triggered pin (continuous, every frame while held), that same event's recolor and IsMoving = true chain off its Started pin (discrete, once per press), and the blink timer lives in a single Event Tick (continuous, every frame) reading a shared IsMoving boolean that the 4 Enhanced Input Action Events' Started/Completed pins set. One Enhanced Input Action Event node can drive different pins for different purposes - don't re-swap the Material every frame off Triggered (no reason to, it already matches), and don't try to drive the blink from an Enhanced Input Action Event at all. That's the Complete Level 3 solution below.

Complete Level 3 solution (all 5 goals combined)

Everything above combined, across BP_Player's own Enhanced Input Action Events and one Event Tick - every Target left unwired throughout, since inside BP_Player's own graph that always means Self:

```
(M_Player_Red, M_Player_Blue, M_Player_Green, M_Player_Yellow,
M_Player_Orange created ahead of time in the Content Browser, each Unlit
with its own Texture Sample wired to Emissive Color)

Enhanced Input Action Event (IA_MoveRight):
  Triggered -> Add Actor World Offset(Self, DeltaLocation = (X = 300 * Get
World Delta Seconds, Y=0, Z=0))
               Get Actor Location -> Break Vector -> Clamp (Float) X and Y
-> Make Vector -> Set Actor Location
               SoundCooldown -= Get World Delta Seconds
               Branch (SoundCooldown <= 0) -> True: Play Sound
2D(sfx_blip.wav); Set SoundCooldown = 0.15
  Started   -> Set Material(Self, Material Index 0, M_Player_Red)
               Set IsMoving = true
  Completed -> Set IsMoving = false

Enhanced Input Action Event (IA_MoveLeft):
  Triggered -> Add Actor World Offset(Self, DeltaLocation = (X = -300 *
Get World Delta Seconds, Y=0, Z=0))
               Get Actor Location -> Break Vector -> Clamp (Float) X and Y
-> Make Vector -> Set Actor Location
```

```
               SoundCooldown -= Get World Delta Seconds
               Branch (SoundCooldown <= 0) -> True: Play Sound
2D(sfx_blip.wav); Set SoundCooldown = 0.15
  Started   -> Set Material(Self, Material Index 0, M_Player_Blue)
               Set IsMoving = true
  Completed -> Set IsMoving = false

Enhanced Input Action Event (IA_MoveUp):
  Triggered -> Add Actor World Offset(Self, DeltaLocation = (X=0, Y = 300
* Get World Delta Seconds, Z=0))
               Get Actor Location -> Break Vector -> Clamp (Float) X and Y
-> Make Vector -> Set Actor Location
               SoundCooldown -= Get World Delta Seconds
               Branch (SoundCooldown <= 0) -> True: Play Sound
2D(sfx_blip.wav); Set SoundCooldown = 0.15
  Started   -> Set Material(Self, Material Index 0, M_Player_Green)
               Set IsMoving = true
  Completed -> Set IsMoving = false

Enhanced Input Action Event (IA_MoveDown):
  Triggered -> Add Actor World Offset(Self, DeltaLocation = (X=0, Y = -300
* Get World Delta Seconds, Z=0))
               Get Actor Location -> Break Vector -> Clamp (Float) X and Y
-> Make Vector -> Set Actor Location
               SoundCooldown -= Get World Delta Seconds
               Branch (SoundCooldown <= 0) -> True: Play Sound
2D(sfx_blip.wav); Set SoundCooldown = 0.15
  Started   -> Set Material(Self, Material Index 0, M_Player_Yellow)
               Set IsMoving = true
  Completed -> Set IsMoving = false

Event Tick ->
  Branch (IsMoving)
    True:  BlinkTimer += DeltaSeconds
           Branch (BlinkTimer >= 0.1) -> True: Blink = NOT Blink;
BlinkTimer = 0
           Set Actor Hidden In Game(Self, Blink)
    False: Set Actor Hidden In Game(Self, false)
           BlinkTimer = 0
           Set Material(Self, Material Index 0, M_Player_Orange)  // idle
```

Alternate Path Notes: C++

Forgetting "EnhancedInput" in Build.cs's PublicDependencyModuleNames is the #1 thing students miss - the code compiles fine (UInputAction/UEnhancedInputComponent are still valid types), but the subsystem silently isn't registered, so nothing fires at runtime. If Enhanced Input C++ 'just doesn't work' with no compile error, check Build.cs first.

The C++ path reuses the exact same IA_MoveRight/Left/Up/Down and IMC_Default assets the Blueprint path built in Setup - Input Actions and Mapping Contexts are just data assets, not tied to either track, which is why a student can freely mix a Blueprint Pawn with C++ input handling or vice versa.

AddMappingContext() in C++ is the code equivalent of the Blueprint path's Add Mapping Context node - both call into the same UEnhancedInputLocalPlayerSubsystem, so they're genuinely the same operation, not two different systems that happen to look similar.

The Level 3 goals map onto C++ calls the same way they map onto Blueprint nodes: SetActorHiddenInGame() in Tick() for the blink, UGameplayStatics::PlaySound2D() for sound, SetMaterial() on the Cube's mesh component (swapping between the same 5 pre-made Material assets used in the Blueprint path) for recolor, and FMath::Clamp() for the boundary check - all called from the same MoveRight()-style functions and a Tick() override.

- Showing both C++ and Blueprint side by side for the same behavior is the clearest way to demystify Blueprints for students who assume "visual" means "not real code" - it's the same engine calls either way, now true of the input layer too since both tracks use Enhanced Input from Level 1 onward instead of diverging.
- This is squarely the optional advanced track - most students should stay on Blueprints; C++ is for students already comfortable with typed languages from Godot, GameMaker, or Unity's C# path.